

Bài báo cáo về Decision Tree

24521901 - Trần Quang Trường

Ngành Khoa học máy tính, Trường Đại học Công nghệ Thông tin, ĐHQG-HCM

Ngày 20 tháng 9 năm 2025

Tóm tắt nội dung:

Trong bài báo cáo này, chúng ta sẽ cùng tìm hiểu về Decision Tree, **công dụng và cách sử dụng**, cũng như trình bày một số **thuật toán xây dựng** Decision Tree và **các tiêu chuẩn split**.

Mục lục

1 Công dụng

2 Cách sử dụng

3 Các thuật toán Decision Tree

3.1 Tiêu chí split

3.1.1 Entropy và Information Gain

3.1.2 Gini Impurity và Gini Gain .

3.2 Thuật toán

3.2.1 ID3 (Iterative Dichotomiser 3)

3.2.2 CART (Classification and Regression Trees)

3.2.3 So sánh ID3 và CART

4 Tổng kết

Giới thiệu

Decision Tree hay cây quyết định là một phương pháp học tập có giám sát (supervised learning) được sử dụng trong các bài toán phân loại (classification) và hồi quy (regression). Trong đó, mục tiêu của phương pháp là tạo ra một mô hình dự đoán được kết quả bằng việc học các quy tắc quyết định (decision rules) từ các **đặc trưng dữ liệu** (data features).

Về bản chất, ý tưởng của Decision Tree là đi tìm ranh giới để phân loại các lớp với nhau (trong bài toán Phân loại) hoặc là một đường để dự đoán giá trị của một biến liên tục (trong bài toán Hồi quy).

Decision Tree rất hữu ích cho phân tích dữ liệu và học máy vì chúng chia nhỏ dữ liệu phức tạp thành

các thành phần dễ quản lý hơn. Bên cạnh đó, Decision Tree có khả năng xử lý nhiều loại dữ liệu khác nhau, từ dữ liệu phân loại rời rạc và không có thứ tự (ví dụ: thích, ghét; nóng, lạnh) cho đến dữ liệu liên tục (như chiều cao, cân nặng).

Quá trình học của cây quyết định có thể được xem như là một chuỗi các bước chia dữ liệu thành những tập con đồng nhất. Cây quyết định bản chất là một cây nhị phân. **Nút gốc** (root node) là đỉnh bắt đầu chứa dữ liệu ban đầu. Thuật toán toán sẽ tìm những đặc trưng và **ngưỡng** (threshold) để chia thành 2 tập con sao cho tốt nhất theo một công thức tính toán nào đó. Quá trình đệ quy sẽ thực hiện việc tìm phép chia tại những nút con đã chia trước đó. Quá trình này sẽ dừng lại khi các đỉnh thuộc một tập con là đồng nhất (tức cùng thuộc một loại) hoặc chiều cao của cây đạt một giới hạn đã định trước. Những nút ở cuối này được gọi là **nút lá** (leaf node).

1 Công dụng

Như đã nói ở phần giới thiệu, ta có thể liệt kê một số công dụng của cây quyết định như sau:

- **Thứ nhất** là trực quan hóa. Ta có thể dễ dàng trực quan hóa cây quyết định.
- **Thứ hai** là khả năng xử lý nhiều loại khác nhau, từ dữ liệu phân loại rời rạc đến dữ liệu có tính liên tục.
- **Thứ ba**, Decision tree không đòi hỏi quá trình tiền xử lý dữ liệu như các mô hình khác. Với những mô hình khác, yêu cầu chuẩn hóa dữ liệu (normalization), hoặc xử lý dữ liệu thiếu (missing data).

2 Cách sử dụng

Ở đây, chúng ta sẽ không đề cập đến Decision tree cho bài toán hồi quy (regression) mà chỉ tập trung vào Decision tree cho bài toán phân loại (classification). Công cụ hỗ trợ là DecisionTreeClassifier trong thư viện scikit-learn của python. Ngoài ra, ta sẽ sử dụng bộ dữ liệu Iris để minh họa cách sử dụng Decision tree.

Toàn bộ đoạn code có thể được truy cập thông qua link google colab [1]

Khai báo các thư viện

Ta cần khai báo (import) các thư viện cần thiết như numpy, matplotlib, seaborn để trực quan hóa dữ liệu.

```
1 import numpy as np
2 import sklearn
```

Lấy dữ liệu

Bộ dữ liệu Iris có sẵn trong thư viện sklearn, ta chỉ việc lấy dữ liệu từ thư viện là được.

```
1 iris = sklearn.datasets.load_iris()
```

Để hiểu rõ hơn về bộ dữ liệu, ta cần xem qua một số thông tin của bộ dữ liệu này:

```
1 iris.feature_names = ['sepal length (cm)',
2                       'sepal width (cm)',
3                       'petal length (cm)',
4                       'petal width (cm)']
```

```
1 iris.target_names = array(['setosa',
2                             'versicolor', 'virginica'],
3                             dtype='<U10')
```

- iris.feature_names là các đặc trưng của hoa.
- iris.target_names là các loại hoa ta cần phân loại.

Ta sẽ gán các đặc trưng vào tập X còn tập cần phân loại vào y:

```
1 X = iris.feature_names
2 y = iris.target_names
```

Train Test Split

Sau khi đã gán các đặc trưng X và tập phân loại y, ta thực hiện bước chia tập thành tập train và tập test (Train test split) với tỉ lệ 8 2 (80% tập train, và 20% tập test).

```
1 from sklearn.model_selection import
   train_test_split
2
3 X_train , X_test , y_train , y_test
   = train_test_split(X , y ,
4 test_size = 0.2 , random_state
   =24521901)
```

Ở đây, ngoài 2 tham số X, y như chúng ta đã biết, còn có hai tham số đó là:

- test_size là tỷ lệ của tập test trong bộ dữ liệu.
- random_state có thể được hiểu là hạt giống (seed). Với mỗi giá trị seed khác nhau sẽ trả về các giá trị khác nhau, nhưng cùng seed (tức cùng giá trị random_state) thì luôn trả về là giống nhau dù chạy bao nhiêu lần.

Sử dụng mô hình

Để sử dụng mô hình, ta cần gọi đến model DecisionTreeClassifier trong thư viện scikit-learn:

```
1 from sklearn.tree import
   DecisionTreeClassifier
2 model = DecisionTreeClassifier(
   max_depth=2, random_state
   =24521901)
```

Huấn luyện mô hình

Ta huấn luyện mô hình bằng cách cho mô hình học dữ liệu từ tập huấn luyện mà ta đã chia trước đó.

```
1 model.fit(X_train, y_train)
```

Ta có thể trực quan cái cây quyết định bằng cách sử dụng export_graphviz:

```
1 from graphviz import Source
2 from sklearn.tree import
   export_graphviz
3
4 # plot tree
```

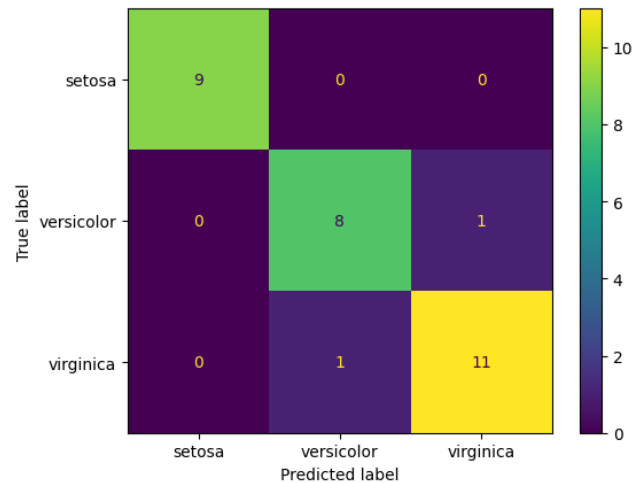
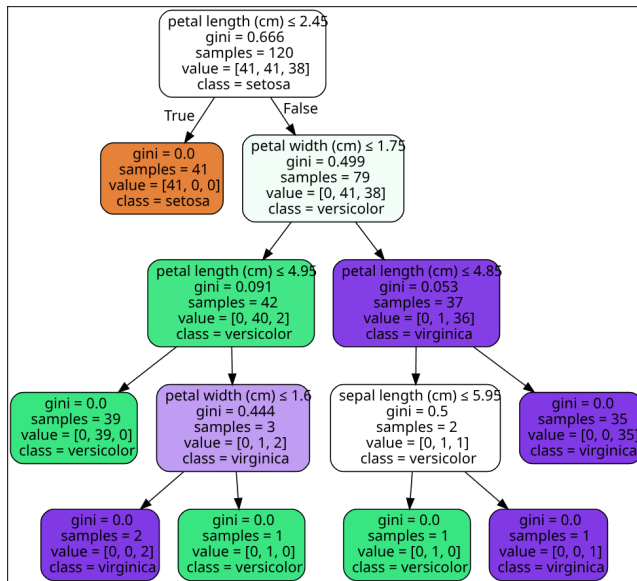
```

5 dot_data = export_graphviz(model,
    out_file=None, feature_names=iris
    .feature_names, class_names=['
    Adelie', 'Chinstrap', 'Gentoo'],
    filled=True, rounded=True,
    special_characters=True)
6 graph = Source(dot_data)
7 graph.render("decision_tree")
8 graph

```

	precision	recall	f1-score	support
Adelie	1.00	1.00	1.00	9
Chinstrap	0.89	0.89	0.89	9
Gentoo	0.92	0.92	0.92	12
accuracy			0.93	30
marco avg	0.94	0.94	0.94	30
weighted avg	0.93	0.93	0.93	30

Ngoài ra, ta cũng có thể trực quan bài toán phân loại bằng biểu đồ dạng bảng:



Đánh giá mô hình

Để đánh giá mô hình, ta sử dụng một số thước đo (metric) để đo độ chính xác của mô hình (Ví dụ: precision, recall, f1-score, support). Có rất nhiều thước đo và ở đây ta sẽ sử dụng hàm `classification_report`. Cụ thể, ta sẽ lấy kết quả dự đoán `y_pred` của mô hình từ tập `X_test`, sau đó đối chiếu kết quả giữa `y_pred` và `y_test` thông qua các thước đo vừa đề cập.

```

1 from sklearn.metrics import
    classification_report
    y_pred = model.predict(X_test)
2
3 print(classification_report(y_test,
    y_pred))

```

Kết quả hiện thị như bảng sau đây:

3 Các thuật toán Decision Tree

Tất cả các thuật toán trong cây quyết định đều yêu cầu các **tiêu chí split** để tách nút. Thuật toán đó thường dựa trên **thuộc tính tốt nhất** (best attribute) để phân chia. Có nhiều tiêu chí phân tách khác nhau dựa trên **tạp chất** (impurity) của một nút. Mục đích chính của tiêu chí phân tách là để giảm tạp chất của nút đó. Dưới đây là 2 phương pháp phổ biến:

3.1 Tiêu chí split

3.1.1 Entropy và Information Gain Entropy lần đầu được giới thiệu trong A Mathematical Theory of Communication của Shannon C.E. [2]. Trong bài viết, Shannon định nghĩa **Entropy** là thước đo impurity của một nút, với công thức:

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2 p_i$$

Trong đó:

- S là tập dữ liệu ban đầu.
- p_i là tỉ lệ các điểm dữ liệu thuộc class i trên tổng số điểm dữ liệu thuộc bộ dữ liệu S .

Information Gain thể hiện sự khác biệt của Entropy trước và sau khi phân tách trên một thuộc tính nhất định. Thuộc tính có Information Gain cao nhất sẽ tạo ra sự phân tách tốt nhất vì nó đang thực hiện công việc tốt nhất trong việc phân loại dữ liệu train theo phân loại mục tiêu của nó. Information Gain cho ta biết mức độ không chắc chắn trong tập dữ liệu được giảm đi bao nhiêu sau khi tách tập dữ liệu đó trên một node cụ thể. Công thức của nó là:

$$IG(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} \cdot Entropy(S_v)$$

Trong đó:

- A là một thuộc tính thuộc tập dữ liệu S
- S_v là tập con S gồm các thuộc tính có $A = v$

3.1.2 Gini Impurity và Gini Gain Theo Breiman et al. (1984) [3], Gini Impurity của tập dữ liệu S được định nghĩa:

$$G(S) = \sum_{c \in C} p_c(1 - p_c) = 1 - \sum_{c \in C} p_c^2$$

Với:

- C là tập nhãn (classes)
- p_c là xác suất xảy ra nhãn c trong S
- $G(S)$ đo impurity trong phân phối nhãn.

Khi tách S theo thuộc tính A thành các tập con $\{S_v\}$ với v là tập giá trị thuộc A . Gini Gain có dạng:

$$GG(S, A) = G(S) - \sum \frac{|S_v|}{|S|} G(S_v)$$

, trong đó:

- S_v là tập con của S gồm các thuộc tính có $A = v$. $GG(S, A)$ đo mức giảm impurity thu được nhờ phân tách theo A .

3.2 Thuật toán

3.2.1 ID3 (Iterative Dichotomiser 3) ID3 là một thuật toán xây dựng Decision Tree được giới thiệu lần đầu bởi Quinlan vào năm 1983[4]. Nó xác định việc phân loại các đối tượng bằng cách kiểm tra các giá trị của các thuộc tính. ID3 xây dựng cây quyết định theo kiểu từ trên xuống (top-down), bắt đầu từ một tập hơn các đối tượng và một đặc điểm đặc biệt. Đây là một thuật toán phân loại theo cách tiếp cận tham lam bằng cách tại mỗi nút chọn một thuộc tính tốt nhất (best attribute) mang lại Information Gain (IG) lớn nhất hoặc Entropy nhỏ nhất. Sau đó, thuộc tính có giá trị IG lớn nhất và Entropy nhỏ nhất sẽ được chọn để tách tập đối tượng. Quá trình này được thực hiện một cách đệ quy cho đến khi tập hợp trong một cây con nhất định là đồng nhất (tức là nó chứa các đối tượng thuộc cùng một loại). Sau đó, nút đó sẽ trở thành nút lá của cây.

Để xây dựng thuật toán ID3, ở mỗi bước, người ta chọn feature tốt nhất để xây dựng Decision Tree. Cụ thể, thuộc tính nào có IG lớn nhất chính là thuộc tính tốt nhất, nghĩa là nó có khả năng phân loại tốt nhất.

Để xây dựng Decision Tree bằng thuật toán ID3, ta thực hiện lần lượt các bước sau:

Algorithm 1 ID3 Decision Tree Algorithm

Require: Training data set $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$
Ensure: Decision tree T

```

1: function ID3( $D$ )
2:   if  $D$  is empty then
3:     return a terminal node with default
       class  $c_{default}$ 
4:   end if
5:   if all instances in  $D$  have same class
       label  $y$  then
6:     return a terminal node with class  $y$ 
7:   end if
8:   if the attribute set  $J$  is empty then
9:     return a terminal node with the
       prevalent class in  $D$ 
10:  end if
11:  Select the feature  $f$  that best splits the
       data using information gain
12:  Create a decision node for  $f$ 
13:  for each value  $b_i$  of  $f$  do
14:    Create a branch for  $b_i$ 
15:    Let  $D_i$  be the subset of  $D$  where  $x_i =$ 
        $b_i$ 
16:    Recursively build the subtree for  $D_i$ 
17:    Attach the subtree to the branch for
        $b_i$ 
18:  end for
19:  return the decision node
20: end function

```

1. Nếu cây quyết định của chúng ta vẫn còn khả năng chia tiếp được hay nói cách khác vẫn tồn tại ít nhất 1 nút không phải là lá, thì ta tính toán Information Gain cho mỗi feature.
2. Tạo nút con S trong cây Decision Tree bằng cách sử dụng feature có Information Gain lớn nhất.
3. Chia các tập samples trong nút trên thành các tập con mà trong đó, mỗi tập con sẽ có giá trị features được chọn ở trên bằng nhau.
4. Nếu tất cả các samples trong một node thuộc cùng 1 class thì node hiện tại sẽ là node lá và mình sẽ có được label của nó.

3.2.2 CART (Classification and Regression Trees) Giới thiệu, ý tưởng thuật toán

CART được giới thiệu lần đầu bởi Breiman et al.[3]. CART là một thuật toán được cải tiến từ ID3, nhưng nó khác ở chỗ nó hỗ trợ cho các bài toán hồi quy (nghĩa là lá của nó dự đoán một số thực chứ không phải một lớp). Khi dùng CART để xây dựng thì cây quyết định của ta có dạng là một cây nhị phân, cụ thể là mỗi nút trong có chính xác 2 cạnh đi ra.

CART xây dựng cây nhị phân bằng cách sử dụng 1 cặp (thuộc tính, ngưỡng (tương ứng với thuộc tính đó)) có giá trị IG lớn nhất tại mỗi nút. CART thường sử dụng Gini để xác định thuộc tính lý tưởng để phân tách. Gini đo tần suất một thuộc tính được chọn ngẫu nhiên bị phân loại sai. Khi đánh giá bằng Gini, giá trị càng thấp càng tốt.

Algorithm 2 CART Algorithm

Require: $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$
Ensure: Decision tree T

```

1: function CART( $D$ )
2:   if  $D$  is empty then
3:     return a terminal node with default
       value or class  $c_{default}$ 
4:   end if
5:   if all instances in  $D$  have the same class
       label  $y$  then
6:     return a terminal node with class  $y$ 
7:   end if
8:   if the feature set  $F$  is empty then
9:     return a leaf node with the average
       value of  $y$  in  $D$ 
10:  end if
11:  Select the best feature  $f$  and split point
        $s$  that minimize the cost function
12:  Create a decision node for  $f$  and  $s$ 
13:  Partition the data set  $D$  into two
       subsets  $D_1$  and  $D_2$  based on the split
14:  Recursively build the subtree for  $D_1$  and
        $D_2$ 
15:  Attach the subtrees to the decision node
16:  return the decision node
17: end function

```

Chi tiết các bước:

1. Tính tất cả các giá trị Information Gain, sau đó chọn node có giá trị IG cao nhất là nút để split.
2. Sau đó, chọn một giá trị threshold tương ứng với feature đó. Chia các samples trong nút đó thành 2 tập, trong đó, tập con thứ nhất lưu các samples có giá trị features $\leq t$, ngược lại, tập con còn lại lưu các samples có giá trị của features đó $> t$.
3. Tạo 2 node con của S, mỗi nút tương ứng với một tập con ở bước trên.
4. Lặp đi lặp lại từ bước (a) cho đến khi các node trong cây đều là lá hoặc thỏa 1 trong các điều kiện trong phần (2). Việc tạo cây sẽ dừng một trong các điều kiện sau được thỏa khi:
 - Không còn mẫu nào chưa được phân tách.
 - Tất cả các mẫu cho một nút thuộc cùng một lớp.
 - Không còn thuộc tính nào để phân tách thêm.

3.2.3 So sánh ID3 và CART Ta có thể tóm tắt ưu dựa vào ưu và nhược điểm.

ID3:

- **Tổng quan:** Thuật toán tham lam, sử dụng Information Gain để chọn thuộc tính. Thích hợp cho dữ liệu phân loại (categorical).
- **Ưu điểm:** Dễ hiểu và dễ cài đặt.
- **Nhược điểm:** Dễ bị quá khớp (overfitting), không phù hợp cho biến liên tục.

CART:

- **Tổng quan:** Sử dụng chỉ số Gini cho phân loại và giảm phương sai cho hồi quy. Thực hiện tách nhị phân.
- **Ưu điểm:** Đa dụng cho cả phân loại và hồi quy; đơn giản hoá mô hình.
- **Nhược điểm:** Có thể bị quá khớp; cần tinh chỉnh và cắt tỉa cây (pruning) cẩn thận.

4 Tổng kết

Qua bài viết, chúng ta đã cùng tìm hiểu về công dụng, cách sử dụng và một số thuật toán của Decision Tree. Tóm lại, Decision Tree là một thuật toán phổ biến vì tính đơn giản. Tuy nhiên, dù có

rất nhiều thuật toán cải tiến nhưng điểm chung của thuật toán Decision Tree là overfitting.

Lời cảm ơn

Bài viết này được lấy ý tưởng từ Bài Decision Tree của trthminh [5] và A Survey of Decision Trees: Concepts, Algorithms, and Applications [6].

Tài liệu

- [1] Trần Quang Trường. Google colab: Decision tree. <https://colab.research.google.com/drive/1DBU6VrdPvy8vm7r3k--oWRWZCt53Bo9z>, 2025.
- [2] C. E. Shannon. A mathematical theory of communication. *Entropy*, 1948.
- [3] Breiman et al. Classification and regression trees. *Gini*, 1984.
- [4] J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1986.
- [5] Trịnh Minh. Decision tree – assignment 1 (cs115.n11.khtn). <https://github.com/trthminh/CS115.N11.KHTN/blob/main/Assignment%201%20-%20Decision%20Tree/21520064%20-%20Decision%20Tree.pdf>, 2022.
- [6] Ibomoiye Domor Mienye and Nobert Rangarirai Jere. A survey of decision trees: Concepts, algorithms, and applications. *IEEE Access*, PP(99), 2024.